OpenAI Responses API Integration for E-Commerce Product Listing Generation

Overview

This project integrates OpenAI's Responses API with vision capabilities to automatically generate structured product listings from images. The pipeline encodes product images as base64 data, sends them alongside a detailed prompt to GPT-4o, and parses the model's response into a Pydantic schema (title, description, features, keywords).

Technical Implementation

The system uses three core components: (1) Base64 image encoding for inline transmission, (2) the Responses API's `client.responses.parse()` method with structured `input_text` and `input_image` content blocks, and (3) Pydantic schema validation to ensure consistent JSON output. The prompt explicitly instructs the model to describe only visually observable details and avoid inventing brand claims or licensing information not verifiable from the image.

Challenges Encountered

The primary challenge was distinguishing between API method signatures. Early iterations confused `client.responses.create()` (unstructured text output) with `client.responses.parse()` (structured Pydantic output), and mixed image input formats from `chat.completions.create()` into the Responses API. The correct format required `input_image` blocks nested within a `role`/`content` message structure, not plain base64 strings embedded in text.

A secondary concern was hallucination risk: without the instruction to ground descriptions only in visible details, the model confidently asserted false claims (e.g., "official Manchester United merchandise"). Adding explicit constraints in the system prompt significantly reduced (though did not eliminate) this risk.

Image resolution also limited detail extraction — product images in the HuggingFace dataset are small thumbnails (300x300 or smaller), constraining the model's ability to observe fine-grained details like stitching, material texture, or precise color gradations.

Output Quality Assessment

Generated listings demonstrate genuine vision-grounding. The Puma shirt description correctly identifies a "two-button placket" visible in the actual image. The Peter England jeans listing accurately describes "subtle fading and whiskering on the thighs and knees." The Titan watch notes "gold-tone accents" and "slender silver hour markers." These are specific enough to be unlikely pure name-based guesses, confirming the model is analyzing pixel data, not just pattern-matching product names.

The Manchester United track pants example shows improved behavior post-prompt refinement: it no longer claims "official merchandise" but instead hedges with "a highlighted logo on the upper leg adds a touch of team spirit" — safer language that avoids unverifiable licensing claims.

Recommendations for Production Use

1. **Higher-Resolution Images**: Use at least 600x600px product photos. Current dataset resolution limits detail extraction.

2. **Confidence Scoring**: Implement a secondary pass where the model rates confidence in each claim (0-1), allowing filtering of potentially hallucinatory details before publication.
3. **Human Review**: For claims with brand/licensing implications, require human approval before listing goes live.
4. **Batch Processing Optimization**: Current 2-second delay between API calls is conservative; reduce to 0.5s for faster throughput, or use async calls.